变量

python中的变量不需要定义类型

```
width = 10
height = 20
area = width * height
print(area)
```

```
200
```

变量类型	符号
Text Type:	`str`
Numeric Types:	`int`,`float`,`complex`
Sequence Types:	`list`,`tuple`,`range`
Mapping Type:	`dict`
Set Types:	`set`,`frozenset`
Boolean Type:	`bool`
Binary Types:	`bytes`,`bytearray`,`memoryview`
None Type:	`NoneType`

```
x = 1j # complex
# 虚部是1，实部是0
# 虚部使用j来拼接
```

字符串

`String`型、`Char`型可以相加，但是不允许直接加整型

```
a = "a"
b = 'b'
ab = a+b
print(ab)
```

```
ab
```

如果希望加某个整型，使用`str()`方法

```
a = 50
b = "Coins"
print(str(a)+" "+b)
```

```
50 Coins
```

字符串重复

```
a = "a"
b = 'b'
result = a*3 + b
print(result)
```

```
aaab
```

使用`.upper()`和`.lower()`方法可以全部大写或者小写

```
a = "My name is Tom."
b = "I love sport"
result = a.upper() + b.lower()
print(result)
```

```python
txt = "We are the so-called \"Vikings\" from the north." # 转义字符

price = 59
txt = f"The price is {price:.2f} dollars"  # 字符插入数据
print(txt)

txt = f"The price is {20 * 59} dollars"    # 字符插入数据
print(txt)
```

列表List

列表是`[]`括起来的Set

创建一个list

```
fruitHub = ["apple","banana","cat"]
print(type(fruitHub))
print(fruitHub)
print(fruitHub[0])
```

```
<class 'list'>
['apple', 'banana', 'cat']
apple
```

创建带有步长的list

```
nums = list(range(0,100,5)) # 0 到 100 步长 5
print(nums)
```

```
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
```

使用`.append()`

```
myList = []
for i in range(10):
    myList.append(i)
print(myList)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

List是引用数据类型：

```
original = [1,2,3]
new = original
new.append(1)
print(original)
print(new)
```

```
[1, 2, 3, 1]
[1, 2, 3, 1]
```

排序数组

```
list = [3,2,4,1]
list.sort()
print(list)
```

```
[1, 2, 3, 4]
```

或者使用`sorted(list)`

切割数组

```
nums = list(range(0,100,5)) # 0 到 100 步长 5
print(nums)
print(nums[1:5:2]) # 下标：[1,5)，步长2
print(nums[1:5]) # 下标：[1,5)
print(nums[10:]) # 下标：[10,length-1)
print(nums[-1]) # 最后一个元素
print(nums[::-1]) # 把集合倒过来
```

```
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
[5, 15]
[5, 10, 15, 20]
[50, 55, 60, 65, 70, 75, 80, 85, 90, 95]
95
[95, 90, 85, 80, 75, 70, 65, 60, 55, 50, 45, 40, 35, 30, 25, 20, 15, 10, 5, 0]
```

切割字符串

```
sentence = "Hello guys, welcome to Python!"
words = sentence.split(" ")
print(words)
```

```
['Hello', 'guys,', 'welcome', 'to', 'Python!']
```

words是一个数组，或者list，可以用for遍历！

获取list下标

```
sentence = "Hello guys, welcome to Python!"
words = sentence.split(" ")
print(words.index("to")) # 3
print(words.index("TOM")) # ValueError: 'TOM' is not in list 报错停止
```

映射

```
x = int(1)   # x will be 1
y = int(2.8) # y will be 2
z = int("3") # z will be 3

x = float(1)     # x will be 1.0
y = float(2.8)   # y will be 2.8
z = float("3")   # z will be 3.0
w = float("4.2") # w will be 4.2
```

元组

tuple 是 `()` 括起来的Set

```
x = ("apple", "banana", "cherry") #tuple
```

运算符

让我们试试运算符

```
a = 2
b = 3
print(a+b)
print(a-b)
print(a*b)
print(a/b) # 除法，得到double/float型
print(a//b) # 除法向下取整，得到int
print(a**b) # power
print(b%a) # 取余
```

```
5
-1
6
0.6666666666666666
0
8
1
```

优先级： `()` > `**` > `*` , `/` > `+` , `-`

比较运算符，和Java相同。 `==` , `!=` , `>` , `<` , `>=` , `<=` ...

逻辑运算符

```
a = 0
b = 1
print(a or b)
print(a and b)
print(not a)
```

```
1
0
True
```

IF 语句

```
x = True
if x:
    print("Executing IF statement")
else:
    print("Executing ELSE statement")
print("Finished")
```

```
Executing IF statement
Finished
```

else if

```
x = False
y = False
if x:
    print("X")
elif y:
    print("Y")
else:
    print("NONE")
```

```
Executing IF statement
Finished
```

For 循环

首先让我们看下`range`！

```
print(list(range(0,5))) # 输出 [0,5), 默认间隔（步长）1
print(list(range(10))) # 默认从0开始，[0,10)
```

```
[0, 1, 2, 3, 4]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
numbers = list(range(4))
for num in numbers:
    square = num ** num
    print(num," ^ 2 =",square) # 可以这样输出！也可以str()
```

```
0  ^ 2 = 1
1  ^ 2 = 1
2  ^ 2 = 4
3  ^ 2 = 27
```

While 循环

```
number = 5
while number > 0:
    print("number = ",number)
    number = number - 1
print("Finished")
```

```
number =  5
number =  4
number =  3
number =  2
number =  1
Finished
```

break语句

```
number = 5
while number > 0:
    if number == 3:
        break
    print("number = ",number)
    number = number - 1
print("Finished")
```

```
number =  5
number =  4
Finished
```

continue 语句

```
number = 5
while number > 0:
    if number == 3:
        number = number - 1
        continue
    print("number = ",number)
    number = number - 1
print("Finished")
```

```
number =  5
number =  4
number =  2
number =  1
Finished
```

函数

一个简单的函数

```
def sum(a,b):
    return a+b
a = 10
b = 2
print(sum(a,b))
```

函数要定义在usage之前！（还记得吗python是解释性语言...所以要定义在之前）

函数不一定要有返回值！但是如果你不加返回值，它就会返回None

```
def person(name,age):
    print(name," is ",age," years old.")
print(person("Tom",23))
```

```
Tom  is  23  years old.
None
```

是的，他会多输出一行None，代表返回值

返回一个String

```
def person(name,age):
    return f"{name.upper()} is {age} years old." # 使用 f-string 格式化字符串
print(person("Tom",23))
```

```
Tom  is  23  years old.
```

默认值

```
def sum(a=0,b=0): #这里=是默认值（如果无传入）
    return a+b
a = 10
b = 2
print(sum())
print(sum(a))
print(sum(b))
print(sum(a,b))
```

```
0
10
2
12
```

默认值的用法

```
def pet(name = None,age = None):
    if name:
        print(f"My pet named {name}")
    if age:
        print(f"My pet aged {age}")
pet(age=3)
```

```
My pet aged 3
```

可以用于判断：在这里我可以检测到没传入name，因此不会输出第三行

非顺序传入

```
def person(name,age):
    return f"{name.upper()} is {age} years old."
print(person(age = 23, name = "tim"))
```

```
TIM is 23 years old.
```

练习：如果小数位>0.5那么向上取整，否则向下取整

```
def autoRound(number):
    if number % 1 > 0.5:
        return number - (number%1) + 1
    else:
        return number - (number%1)

a = 3.4
b = 3.6
print(autoRound(a))
print(autoRound(b))
```

```
3.0
4.0
```

返回多个值

```
def findSpecialValue(myList):
    maxValue = max(myList)
    minValue = min(myList)
    return maxValue,minValue
numbers = [1,4,7]
print(findSpecialValue(numbers))
```

```
(7, 1)
```

这里的返回可以认为是一个数组...可以使用索引!

可变参数

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

`*kids` 是一个特殊的参数形式，被称为可变参数（也叫不定长参数）。

`*` 这个星号的作用是让函数能够接受任意数量的位置参数，这些参数会被收集到一个元组（`tuple`）里，元组的名称是 `kids`。

也就是说，调用函数时可以传入任意多个参数，这些参数都会被放到 `kids` 这个元组中。

Python自带函数：Built-in Functions — Python 3.9.21 documentation

	Built-in Functions			
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()

	Built-in Functions			
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

字典

```
x = {"name" : "John", "age" : 36} #dict
```

字典是 `{}` 括起来的Map

```
month = {
    1: "Apple",
    2: "Banana",
    3: "Cat",
}
print(month.values())
print(month.keys())
print(month)
print(month.get(1)) # 传入Key得到Value
print(month.pop(1)) # pop 后会返回Value
print(month.get(1)) # 传入Key得到Value
```

```
dict_values(['Apple', 'Banana', 'Cat'])
dict_keys([1, 2, 3])
{1: 'Apple', 2: 'Banana', 3: 'Cat'}
Apple
Apple
None
```

实际上是键值对...

使用update语句更新、增删数据

```
month = {
    1: "Apple",
    2: "Banana",
    3: "Cat",
}
month.update({4:"Fish"})
print(month)
month.update({2:"Horse"})
print(month)
```

```
{1: 'Apple', 2: 'Banana', 3: 'Cat', 4: 'Fish'}
{1: 'Apple', 2: 'Horse', 3: 'Cat', 4: 'Fish'}
```

集合

集合是`{}`括起来的Set

```
x = {"apple", "banana", "cherry"} #set
```

Random 函数

```
import random as rand
a = rand.random() # 输出 [0,1)
print(a)
```

```
0.28724590889882673
```

递归

简单的递归，例如斐波那契数列

```
def fibonacci(currentIndex):
    if currentIndex == 1 or currentIndex == 2:
        return 1
    else:
        return fibonacci(currentIndex-1) + fibonacci(currentIndex-2)

print(fibonacci(20))
```

```
6765
```

特殊

Bytes

```
x = b"Hello"
```

- `b` 前缀：这个前缀告知 Python 解释器，后面的字符串将被视为二进制数据，而不是普通的 Unicode 字符串。
- `"Hello"`：这是一个由 ASCII 字符组成的字符串。在 ASCII 编码中，每个字符都可以用一个字节（8 位）来表示。因此，`b"Hello"` 实际上是一个包含 5 个字节的 `bytes` 对象，每个字节对应一个字符的 ASCII 码值。

Bytearray

在 Python 里，`bytearray` 是一种可变的序列类型，它由 0 到 255 范围内的整数组成，用来表示二进制数据。和 bytes 类型不同，`bytearray` 可以进行修改，例如添加、删除或替换元素。

```

# 创建一个长度为 5 的 bytearray，初始元素值都为 0
x = bytearray(5)
print("初始的 bytearray:", x)

# 修改 bytearray 中的元素
x[0] = 65  # ASCII 码中 65 代表 'A'
print("修改后的 bytearray:", x)

# 将 bytearray 转换为字符串
string_from_bytearray = x.decode('utf-8', errors='ignore')
print("转换后的字符串:", string_from_bytearray)

```

上述代码先是创建了一个长度为 5 的 `bytearray` 对象，接着修改了其中一个元素的值，最后尝试把 bytearray 转换为字符串。

Memoryview

```
x = memoryview(bytes(5)) #memoryview
```

None

```
x = None #NoneType
```

Pandas

Series

把List转为Series

```
import pandas
data = pandas.Series([1,2,3,4])
print(data)
```

```
0    1
1    2
2    3
3    4
dtype: int64
```

`.values`和`.index`

```
import pandas
data = pandas.Series([1,2,3,4])
print(data.values)
print(data.index)
```

```
[1 2 3 4]
RangeIndex(start=0, stop=4, step=1)
```

Series 可以像字典一样...

```
import pandas
data = pandas.Series(data = [0,1,2,3,False],index=["a",3,True,"D","E"])
print(data.get("a"))
print(data.get(True))
print(data.get(3))
```

```
0
2
1
```

这个时候`dtype`就是`object`

Series当然，也可以储存list

```
import pandas
obj = pandas.Series(index = ["cities","postCode"],
                    data= [["shanghai","beijing"],["001","002"]])
print(obj.get("cities")[0]," postCode =  ",obj.get("postCode")[0])
```

```
shanghai  postCode =   001
```

DataFrame

首先，让我们创建一个字典

```
data = {
    "cities" : ["Shanghai","Beijing"],
    "postCode" : ["001","002"],
    123 : 456
}
print(data)
```

```
{'cities': ['Shanghai', 'Beijing'], 'postCode': ['001', '002'], 123: 456}
```

然后把字典转为`pandas`下的DataFrame

```
import pandas
data = {
    "cities" : ["Shanghai","Beijing"],
    "postCode" : ["001","002"],
    123 : [456,789]
}
frame = pandas.DataFrame(data)
print(frame)
```

```
     cities postCode  123
0  Shanghai      001  456
1   Beijing      002  789
```

当然我们可以进一步自定义这个序号...

```
import pandas
data = {
    "cities" : ["Shanghai","Beijing","Guangdong"],
    "postCode" : ["001","002","003"],
    "year" : [1999,1998,2001],
    "position" : ['E','N','S']
}
frame = pandas.DataFrame(data,index=["No.1","No.2","No.3"])
print(frame)
```

```
         cities postCode  year position
No.1   Shanghai      001  1999        E
No.2    Beijing      002  1998        N
No.3  Guangdong      003  2001        S
```

可以选择某些列

```python
import pandas
data = {
    "cities" : ["Shanghai","Beijing","Guangdong"],
    "postCode" : ["001","002","003"],
    "year" : [1999,1998,2001],
    "position" : ['E','N','S']
}
frame = pandas.DataFrame(data = data,
                         index=["No.1","No.2","No.3"],
                         columns=["cities","postCode"]) # 这里只选了两列
print(frame)
```

```
         cities postCode
No.1   Shanghai      001
No.2    Beijing      002
No.3  Guangdong      003
```

更改某些列

```python
import pandas
data = {
    "cities" : ["Shanghai","Beijing","Guangdong"],
    "postCode" : ["001","002","003"],
    "year" : [1999,1998,2001],
    "position" : ['E','N','S']
}
frame = pandas.DataFrame(data = data,
                         index=["No.1","No.2","No.3"],
                         columns=["cities"])
frame["cities"] = ["A","B","C"]
print(frame)
```

```
    cities
No.1      A
No.2      B
No.3      C
```

Numpy

转矩阵

```
import numpy
import pandas
numbers = numpy.arange(9) # [0,9)步长1
data = numpy.reshape(numbers,(3,3)) # 转为 3*3 矩阵
print(data)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

转为DataFrame

```
import numpy
import pandas
numbers = numpy.arange(9) # [0,9)步长1
data = numpy.reshape(numbers,(3,3)) # 转为 3*3 矩阵
frame = pandas.DataFrame(data = data,
                         columns= ["colA","colB","colC"],
                         index = ["No.1","No.2","No.3"])
print(frame)

```

```
      colA  colB  colC
No.1     0     1     2
No.2     3     4     5
No.3     6     7     8
```

删除DataFrame的 Row 或者 Column

删除Row

```
import numpy
import pandas
numbers = numpy.arange(9) # [0,9)步长1
data = numpy.reshape(numbers,(3,3)) # 转为 3*3 矩阵
frame = pandas.DataFrame(data = data,
                         columns= ["colA","colB","colC"],
                         index = ["No.1","No.2","No.3"])

frame = frame.drop(labels="No.1",axis=0)
print(frame)
```

```
      colA  colB  colC
No.2     3     4     5
No.3     6     7     8
```

删除Column

```
import numpy
import pandas
numbers = numpy.arange(9) # [0,9)步长1
data = numpy.reshape(numbers,(3,3)) # 转为 3*3 矩阵
frame = pandas.DataFrame(data = data,
                         columns= ["colA","colB","colC"],
                         index = ["No.1","No.2","No.3"])

frame = frame.drop(labels="colA",axis=1)
print(frame)
```

```
      colB  colC
No.1     1     2
No.2     4     5
No.3     7     8
```

当然，写成 `frame = frame.drop(labels="colA",axis="columns")` 这样更好阅读！！

如果我们想要一次删除多行、多列呢，只需要传入list即可

```
import numpy
import pandas
numbers = numpy.arange(9) # [0,9)步长1
data = numpy.reshape(numbers,(3,3)) # 转为 3*3 矩阵
frame = pandas.DataFrame(data = data,
                         columns= ["colA","colB","colC"],
                         index = ["No.1","No.2","No.3"])

frame = frame.drop(labels=["colA","colB"],axis="columns")
print(frame)
```

```
      colC
No.1     2
No.2     5
No.3     8
```

非固定索引获取一个或多个column、row

```
import numpy
import pandas
numbers = numpy.arange(9) # [0,9)步长1
data = numpy.reshape(numbers,(3,3)) # 转为 3*3 矩阵
frame = pandas.DataFrame(data = data,
                         columns= ["colA","colB","colC"],
                         index = ["No.1","No.2","No.3"])


print(frame["colA"]) # 获取1个column
print("----------------------")
print(frame[["colA","colB"]]) # 获取多个column，这里有两层括号
print("----------------------")
print(frame.loc["No.1"])  # 获取一个row
print("----------------------")
print(frame.loc["No.1":"No.3"])  # 获取多个row
```

```
No.1    0
No.2    3
No.3    6
Name: colA, dtype: int64
----------------------
      colA  colB
No.1     0     1
No.2     3     4
No.3     6     7
----------------------
colA    0
colB    1
colC    2
Name: No.1, dtype: int64
```

```
----------------------
      colA  colB  colC
No.1     0     1     2
No.2     3     4     5
No.3     6     7     8
```

固定索引获取一个、多个column、row

```python
print(frame.iloc[0:]) # 获取第一个（索引0）row
print("----------")
print(frame.iloc[::-1]) # 获取 倒过来的row

print("--------------------")
print(frame.iloc[:,0:]) # 获取所有row（:）但是 0 column
print(frame.iloc[:,0:2]) # 获取所有row（:）但是 [0,2) column
```

```
      colA  colB  colC
No.1     0     1     2
No.2     3     4     5
No.3     6     7     8
----------
      colA  colB  colC
No.3     6     7     8
No.2     3     4     5
No.1     0     1     2
--------------------
      colA  colB  colC
No.1     0     1     2
No.2     3     4     5
No.3     6     7     8
      colA  colB
No.1     0     1
No.2     3     4
No.3     6     7
```

截取矩阵

我们现在有这样的DataFrame

```
      colA  colB  colC
No.1     0     1     2
No.2     3     4     5
No.3     6     7     8
```

那么如何截取右下角四个呢?

```
print(frame.iloc[1:3,1:3])
```

```
      colB  colC
No.2     4     5
No.3     7     8
```

当然除了`iloc`，也可以使用非固定索引`loc`完成！

配合上赋值，就可以操作这些row和column了

```
myData = frame.loc["No.1"]
print("----------------------")
for i in myData:
    print(i)
```

矩阵相加

```
import numpy
import pandas
df1 = pandas.DataFrame(numpy.arange(3*4).reshape(3,4),
                       index = ["row1","row2","row3"],
                       columns=["c1","c2","c3","c4"])
df2 = pandas.DataFrame(numpy.arange(3*4).reshape(3,4),
                       index = ["row1","row2","row3"],
                       columns=["c3","c4","c5","c6"])
print(df1)
print("----------")
print(df2)
print("----------")
print(df1+df2)
print("----------")
print(df1.add(df2,fill_value=0))
```

```
      c1  c2  c3  c4
row1   0   1   2   3
row2   4   5   6   7
row3   8   9  10  11
----------
      c3  c4  c5  c6
row1   0   1   2   3
row2   4   5   6   7
row3   8   9  10  11
----------
      c1  c2  c3  c4  c5  c6
row1 NaN NaN   2   4 NaN NaN
```

```
row2 NaN NaN  10  12 NaN NaN
row3 NaN NaN  18  20 NaN NaN
----------
       c1   c2  c3  c4    c5    c6
row1  0.0  1.0   2   4   2.0   3.0
row2  4.0  5.0  10  12   6.0   7.0
row3  8.0  9.0  18  20  10.0  11.0
```

如果我们直接相加，因为一些 column对不上，因此不存在的值就是NaN，然后就无法想加

但是我们可以使用`.add()`语句来填充不存在的值NaN，这样就可以相加了

计算矩阵row、column的总和、均值

```python
import numpy
import pandas
df1 = pandas.DataFrame(numpy.arange(3*4).reshape(3,4),
                       index = ["row1","row2","row3"],
                       columns=["c1","c2","c3","c4"])
df2 = pandas.DataFrame(numpy.arange(3*4).reshape(3,4),
                       index = ["row1","row2","row3"],
                       columns=["c3","c4","c5","c6"])

df3 = df1.add(df2,fill_value=0)
print(df3)
print("====================")
print(df3.sum(axis = "columns")) # column sum
print("====================")
print(df3.sum()) # row sum
```

```
       c1   c2  c3  c4    c5    c6
row1  0.0  1.0   2   4   2.0   3.0
row2  4.0  5.0  10  12   6.0   7.0
row3  8.0  9.0  18  20  10.0  11.0
====================
row1    12.0
row2    44.0
row3    76.0
dtype: float64
====================
c1    12.0
c2    15.0
c3    30.0
c4    36.0
c5    18.0
c6    21.0
dtype: float64
```

矩阵均值和add方法相同...

```
import numpy
import pandas
df1 = pandas.DataFrame(numpy.arange(3*4).reshape(3,4),
                       index = ["row1","row2","row3"],
                       columns=["c1","c2","c3","c4"])
df2 = pandas.DataFrame(numpy.arange(3*4).reshape(3,4),
                       index = ["row1","row2","row3"],
                       columns=["c3","c4","c5","c6"])

df3 = df1.add(df2,fill_value=0)
print(df3)
print("====================")
print(df3.mean(axis = "columns")) # column sum
print("====================")
print(df3.mean()) # row sum
```

```
       c1   c2  c3  c4    c5    c6
row1  0.0  1.0   2   4   2.0   3.0
row2  4.0  5.0  10  12   6.0   7.0
row3  8.0  9.0  18  20  10.0  11.0
====================
row1     2.000000
row2     7.333333
row3    12.666667
dtype: float64
====================
c1     4.0
c2     5.0
c3    10.0
c4    12.0
c5     6.0
c6     7.0
dtype: float64
```

计算全矩阵总和、总均值

```
print(df3.sum().sum()) # 全部总和
print(df3.mean().mean()) # 全部总均值
```

更多矩阵详细数据

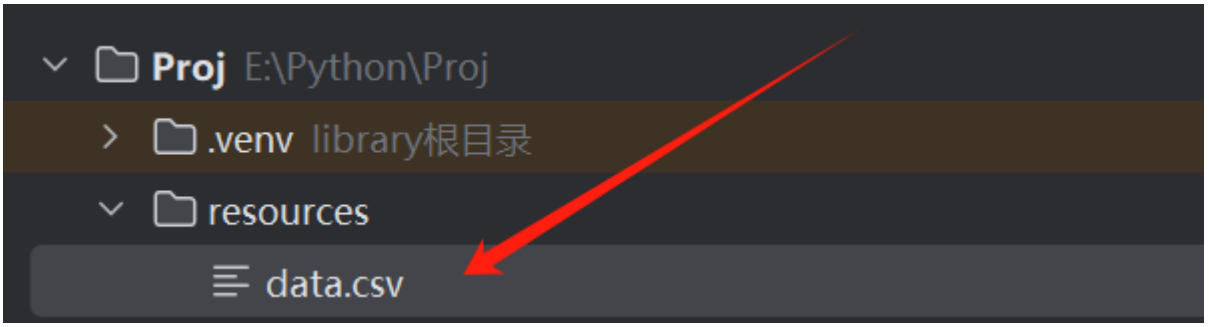
使用`.describe()`

```
print(df3.describe())
```

```
        c1   c2    c3    c4    c5    c6
count  3.0  3.0   3.0   3.0   3.0   3.0
mean   4.0  5.0  10.0  12.0   6.0   7.0
std    4.0  4.0   8.0   8.0   4.0   4.0
min    0.0  1.0   2.0   4.0   2.0   3.0
25%    2.0  3.0   6.0   8.0   4.0   5.0
50%    4.0  5.0  10.0  12.0   6.0   7.0
75%    6.0  7.0  14.0  16.0   8.0   9.0
max    8.0  9.0  18.0  20.0  10.0  11.0
```

导入.CSV

```python
import numpy
import pandas
csvFile = pandas.read_csv("resources/data.csv")
print(csvFile.loc[0]) # 打印第一个row数据
```



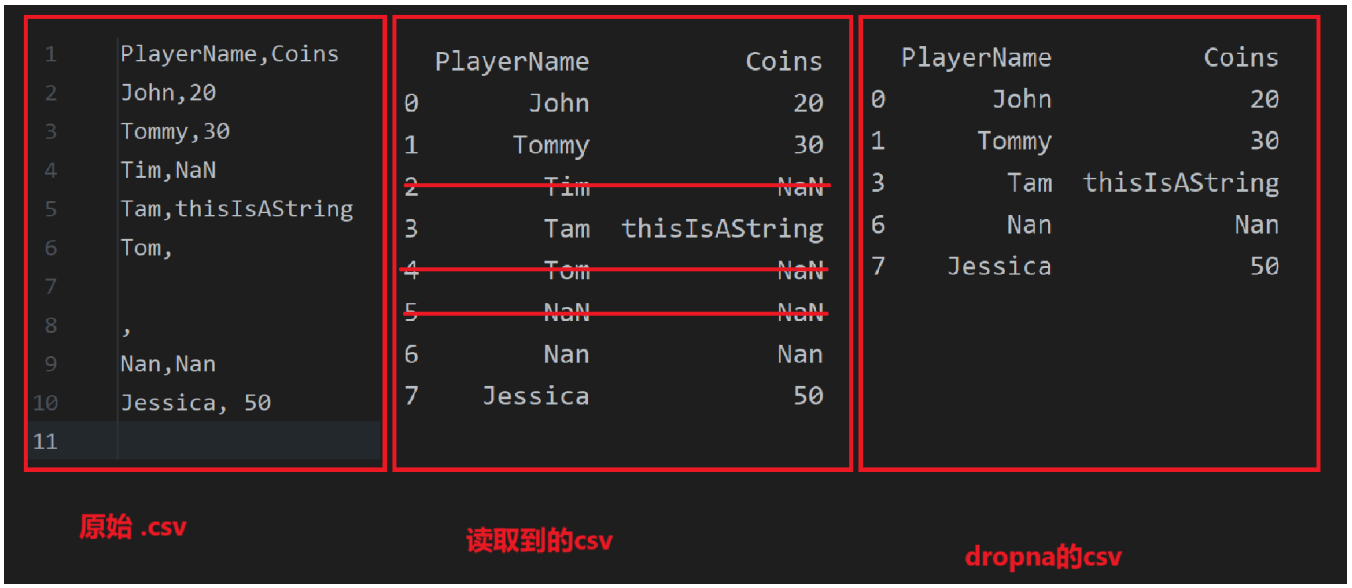

```
Name,Age
Tom,23
John,24
Tommy,25
Walter White,50
```

.CSV数据的第一行是说明，第一个数据实际上在第二行

导入进来的数据实际上是 DataFrame格式，因此可以用`loc`和`iloc`方法

去除NaN数据

```
csvFile = csvFile.dropna()
```



实际上，只要你的Value是`NaN`，他就会剔除，包括`None`、`null`也是

当然，如果使用`csvFile = csvFile.dropna(how="all")`，那么一个row只有所有参数为 `NaN`、`None`、`null` 才会被删除

使用`.fillna(0)`把上述空数据填充为0

去除重复数据

```python
import pandas
csvFile = pandas.read_csv("resources/data.csv")

print(csvFile)
print(csvFile.drop_duplicates())
```

```
   PlayerName  Coins
0           1      1
1           2      3
2           3      2
3           4      1
4           1      1
   PlayerName  Coins
0           1      1
1           2      3
2           3      2
3           4      1
```

`print(csvFile.drop_duplicates(keep="last"))` 会删除所有的重复行，只保留最后一个重复行